

MAP SIDE AND REDUCE SIDE JOINS

Joins:-

=====

Joins is one of the interesting features available in MapReduce. Joins performed by Mapper are called as Map-side Joins. Joins performed by Reducer can be treated as Reduce-side joins. Frameworks like Pig, Hive, or Cascading has support for performing joins.

Before diving into the implementation let us understand the problem thoroughly. If we have two datasets, for example, one dataset having user ids, names and the other having the user activity over the application. In-order to find out which user have performed what activity on the application we might need to join these two datasets such as both user names and the user activity will be joined together.

Join can be applied based on the dataset size if one dataset is very small to be distributed across the cluster then we can use Side Data Distribution technique.

Side Data Distribution:-

=====

Side-Data is the additional data needed by the job to process the main dataset. The critical part is to make this side-data available to all the map or reduce tasks running in the cluster. It is possible to cache the side-data in memory in a static field, so that the tasks running successively in a task tracker will share the data. Using the Task JVM re-use feature we can handle this. When using this feature we should make sure that the amount of memory needed to cache the data should not affect the memory needed for Shuffle and sort phase.

Caching of side data can be done in two ways,

Job Configuration:- Using Job Configuration object setter method we can set the key-value pairs and the same can be retrieved in the map or reduce tasks. We should be careful using this option to not to use huge amount of data to be shared in this way since the configuration is read by the JobTracker, TaskTracker and the child JVMs and everytime the configurations will be loaded into the memory.

Distributed Cache:-

=====

Side-Data can be shared using the Hadoop's Distributed cache mechanism. We can copy files and archives to the task nodes when the tasks need to run. Usually this is the preferable way over the JobConfiguration.

If both the datasets are too large then we cannot copy either of the datasets to each node in the cluster as we did in the Side data distribution.

We can still join the records using MapReduce with a Map-side or reduce-side joins.

Map-Side Joins:-

=====

The inputs for to each map must be partitioned and sorted in a specific way. Each input dataset must be divided into the same number of partitions, and it must be sorted by the same key (the join key) in each source. All the records for a particular key must reside in the same partition and which is mandatory. A map-side join can be used to join the outputs of several jobs that had the same number of reducers, the same keys, and output files that are not splittable which means the output files should not be bigger than the HDFS block size.

Using the `org.apache.hadoop.mapred.join.CompositeInputFormat` class we can achieve this. The join type (Inner or Outer) is configurable using the join expression. for ex,
`func := tbl(<class>,"<path>");`
We can set it to the `CompositeInputFormat` using,

```
inner(tbl(org.apache.hadoop.mapreduce.lib.input.SequenceFileInputFormat.class,
"hdfs://localhost:8000/usr/data"),
```

To search type and hit enter

RECENT POSTS

[Beringea Leads Series B Funding Round for Big Data Partnership, EMEA's Leading Big Data Services Company](#)
[So, do i need hiking boots for my Big Data Journey?](#)
[The Big Data Heartbeat](#)
[Big Data is a truly global phenomenon](#)
[Getting into the Christmas Spirit](#)

ARCHIVES

Select Month ▼

CATEGORIES

Select Category ▼

NEWSLETTER

Enter Email

Subscribe

```
tbl(org.apache.hadoop.mapreduce.lib.input.SequenceFileInputFormat.class,  
"hdfs://localhost:8000/usr/activity");
```

We can achieve following kind of joins using Map-Side techniques,

- 1) Inner Join
- 2) Outer Join
- 3) Override – MultiFilter for a given key, preferred values from the right most source

Reduce-Side Joins:-

=====

Reduce-Side joins are more simple than Map-Side joins since the input datasets need not to be structured. But it is less efficient as both datasets have to go through the MapReduce shuffle phase. the records with the same key are brought together in the reducer. We can also use the Secondary Sort technique to control the order of the records.

Posted on February 29, 2012 in [Apache Hadoop](#), [Blog](#), [Hadoop Common](#), [Hadoop Ecosystem](#), [MapReduce](#), [Science](#), [Technology](#), [Training](#)

🔗 CONCEPT, 🔗 HADOOP, 🔗 JOIN EXPRESSIONS, 🔗 JOINS, 🔗 JOINS IN MAPREDUCE, 🔗 MAP-SIDE JOIN, 🔗 MAP-SIDE JOINS, 🔗 MAPREDUCE FEATURES, 🔗 REDUCE-SIDE JOIN, 🔗 SIDE-DATA, 🔗 TECHNOLOGY

[“Introducing YARN” – Hadoop No More a Baby Elephant →](#)

[← Bloom Filter Vs Feature Hashing](#)

Share the Story

RESPONSE (1)

Praveenesh

April 30, 2012 at 10:21 am · [Log in to Reply](#)

Very high level overview of how joining happens in Pig 0.8 onwards:

```
If ( Input can fit into memory):  
  if (no outer join on small input):  
    Replicated Join  
if ( Both input data can be sorted on the join key )  
  if (no outer join)  
    Merge Join  
  else if ( no skewed Join key)  
    Default Join  
  else  
    Skewed Join
```

PS – Default, Skewed, Merge and Replicated are the types of Join Pig supports.

LEAVE A REPLY

You must be [logged in](#) to post a comment.



© 2014 Big Data Partnership Limited. All Rights Reserved.

Big Data Partnership Limited is a company registered in England & Wales with Company No 7904824
Registered Office: Finsgate, 5-7 Cranwood Street, London, EC1V 9EE
[Terms of Use](#) | [Privacy Policy](#) | [Trademark](#)

This site uses cookies [Learn More](#)